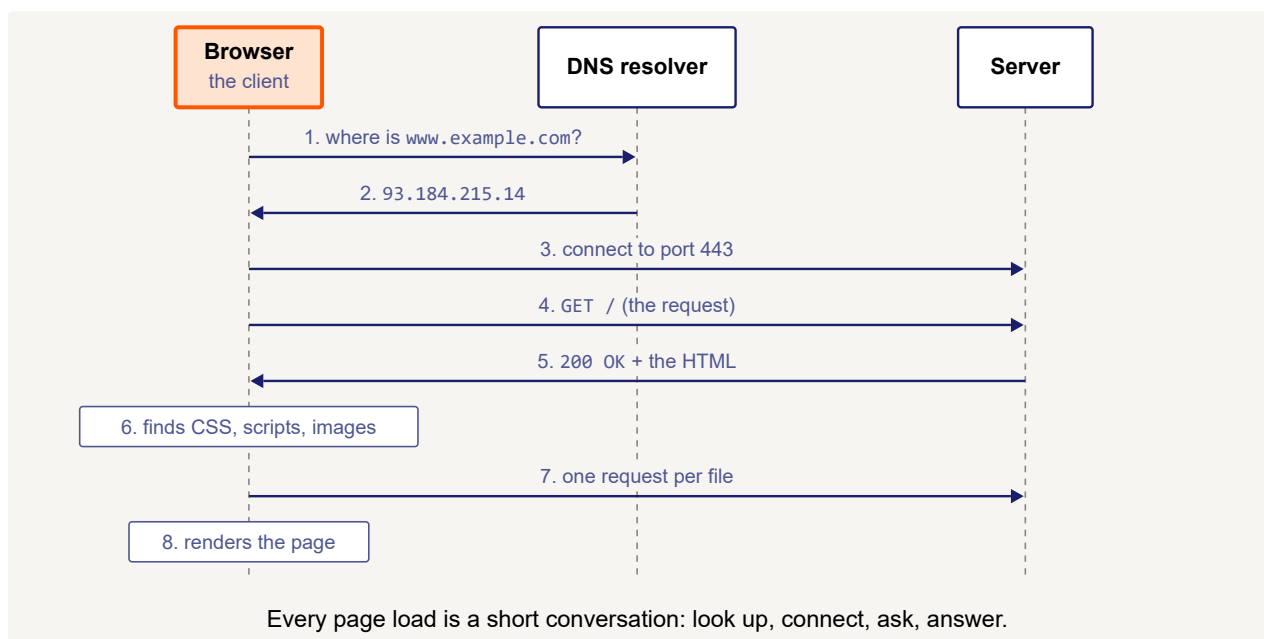


1. **From address to page:** Client and server: a conversation · Anatomy of a URL · DNS: from names to addresses · HTTP and HTTPS
2. **HTTP messages:** HTTP methods · Inside an HTTP request · Inside an HTTP response
3. **Status codes:** Status code families · The status codes you will use
4. **In the browser: rendering and DevTools:** From bytes to pixels: how the browser renders · Watching the conversation: the Network tab
5. **The big picture:** Front end and back end · Static and dynamic · From static pages to a full-stack app

From address to page

Client and server: a conversation

The web is a conversation between two programs: a **client** (usually the browser) sends a **request**, and a **server** answers with a **response**.



Client

The program that starts the conversation: usually the browser, but also command-line tools such as `curl` and your own JavaScript.

Server

A program that waits for requests, decides how to answer each one and sends a response. The word also names the computer that runs it.

Request and response

Always a pair, and the client always speaks first.

Example. You open `https://www.example.com/`. The browser requests `/` and gets HTML that mentions `styles.css`, `logo.png` and `app.js`, so it sends three more requests and gets three more responses: **four** request/response pairs for one page. A real site often needs fifty or more.

Common mistake. Thinking the server sends updates on its own. In plain HTTP the server only ever **answers**: if what you see changes, the browser asked again (you reloaded, or JavaScript requested new data).

Anatomy of a URL

A **URL** (Uniform Resource Locator) is the address of one resource: how to talk (scheme), to which server (host and port), what to ask for (path), with which options (query string) and where to look inside the result (fragment).

The parts of `https://shop.example.com:8443/products/42?color=red&size=m#reviews`

Part	Value here	What it does
------	------------	--------------

Part	Value here	What it does
Scheme	https	The protocol; https is HTTP over an encrypted connection
Host	shop.example.com	The server's name; DNS turns it into an IP address
Port	8443	Which program on that machine
Path	/products/42	Which resource on that server
Query string	color=red&size=m	Options, as key=value pairs after ?
Fragment	reviews	A position inside the resource; never sent

Port Usually left out, because each scheme has a default: **443** for https, **80** for http. You see it while you develop: in `http://localhost:3000`, `localhost` is the name of your own computer and `3000` the port your server listens on.

Query string Starts with `?` and holds `key=value` pairs joined by `&`. Characters not allowed in a URL are **percent-encoded**: a space becomes `%20`, `@` becomes `%40`.

Fragment After `#`. The browser keeps it and scrolls to the element whose `id` matches.

Example. In `https://maps.example.com/search?q=madrid&zoom=12#results` the browser contacts `maps.example.com` on port `443` (the https default) and asks for `/search?q=madrid&zoom=12`. Changing `zoom=12` to `zoom=15` asks for the same resource with another option. `#results` is never sent: the browser uses it afterwards to scroll to `id="results"`.

Common mistake. Writing a link without the scheme: `<a href="www.example.com">`. Without `https://` the browser reads the value as a **relative path** and looks for a file called `www.example.com` in the current folder, which gives a `404`. External links need the full absolute URL.

DNS: from names to addresses

The **DNS (Domain Name System)** translates a host name such as `www.example.com` into the **IP address** of its server, like a phone book for the internet.

- **The resolver answers:** a DNS server normally run by your network or internet provider.
- **Cached:** the answer is kept for a while, so the next visit skips the lookup.
- **Only the host** takes part: the path, query string and fragment play no role.
- `localhost` always means this same computer (address `127.0.0.1`): it is how you reach a server running on your own laptop.

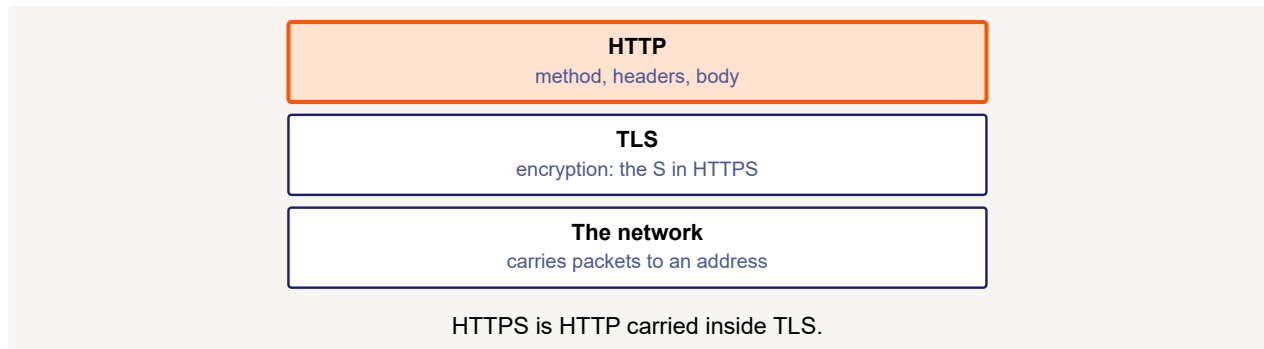
Example. For `https://www.example.com/courses`, the DNS query asks only for `www.example.com` and gets an IP address. The browser connects to it on port `443` and asks for `/courses`, naming the site in a `Host` header, because one IP address can host many sites.

Common mistake. Calling every failure "a 404". If the browser says the server's address could not be found (Chrome shows `DNS_PROBE_FINISHED_NXDOMAIN`), the lookup failed and **no request was ever sent**, so there is no status code. A `404` is the opposite: the server was found and answered "I do not have that resource" (see [Status code families](#)).

HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the agreed format of the request and response messages between client

and server; **HTTPS** is the same protocol sent through an encrypted connection.



Message format In HTTP/1.1 a message is readable text: a **first line, headers** (one Name : value per line), an **empty line** and an optional **body**. HTTP/2 and HTTP/3 pack the same information in binary, and DevTools shows them the same way.

Stateless The server handles each request on its own and does not remember earlier ones. A site "remembers" you only because the browser sends an identifier, a **cookie** or a **token**, with every request (see [Remembering who you are](#)).

HTTPS HTTP inside **TLS** encryption: nobody on the way (a café Wi-Fi) can read or change the messages, and the server proves its identity with a certificate. The padlock means the connection is private, not that the site is honest.

Example. HTTP is a postcard: every network it passes through can read it; HTTPS seals it in an envelope. A login form sent over `http://` on a café network travels as readable text; over `https://` the network only sees that you talked to that server.

Common mistake. Believing the server "knows" it is you because you visited a second ago. Without a cookie or token sent again, your second request is as anonymous as your first: that is what stateless means.

HTTP messages

HTTP methods

The **method** (or verb) is the first word of a request and states the intent: **read** (GET), **create or submit** (POST), **replace** (PUT), **partially update** (PATCH) or **remove** (DELETE).

The five everyday methods, on a to-do list

Method	Intent	Body?	Example
GET	Read a resource or a list	No	GET /tasks/7 returns task 7
POST	Create a resource, submit data	Yes	POST /tasks with a new task creates task 8
PUT	Replace a resource completely	Yes	PUT /tasks/7 with the full new version
PATCH	Change some fields	Yes	PATCH /tasks/7 with {"done":true}
DELETE	Remove a resource	Usually no	DELETE /tasks/7

Safe GET should not change anything on the server, so browsers, caches and search engines may repeat or prefetch it freely.

Idempotent The same final effect whether sent once or three times: GET, PUT, DELETE. POST is neither: sent twice, it usually creates two things.

Example. A user ticks task 7 as done. The front end sends PATCH /tasks/7 with the body {"done":true}: only that field changes. Sending PUT /tasks/7 with {"done":true} would **replace** the task with an object that has no title any more, because PUT means "this is the whole new version".

Common mistake. Using GET for actions that change data, such as a link `<a href="/tasks/7/delete">`. Browsers prefetch links, and crawlers and chat-app link previews follow them automatically, so a bot could delete your data. Changes must use POST, PUT, PATCH or DELETE.

Inside an HTTP request

An HTTP request is made of a **request line** (method, path, HTTP version), **headers**, an empty line and an optional **body**.

Host	Which site: one IP address can serve many.
User-Agent	Which browser or tool sent it.
Accept	Which response formats the client prefers.
Content-Type	The format of the body, when there is one.
Cookie, Authorization	Who you are.

HTTP

```
POST /api/login HTTP/1.1
Host: shop.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Content-Type: application/json
Content-Length: 48

{"email":"ana@example.com","password":"s3cret!"}
```

Example. Read the sample request line by line: the **request line** asks to create something at `/api/login` with HTTP/1.1; Host names the site; Accept asks for a JSON answer; Content-Type and Content-Length announce a JSON body of 48 bytes; after the **empty line** comes the **body** with the login data.

Common mistake. Putting secrets in the URL of a GET request, as in `/login?password=s3cret!`. URLs are saved in browser history, bookmarks and server logs. Sensitive data belongs in the **body** of a POST request, sent over HTTPS.

Inside an HTTP response

An HTTP response is made of a **status line** (HTTP version, status code, reason phrase), **headers**, an empty line and usually a **body**.

Content-Type	How to read the body: text/html is shown as a page, text/css applied as styles, application/json treated as data. The most important response header.
Content-Length	The size of the body in bytes.
Location	Where to go next: used by redirects and by 201 Created.
Set-Cookie	Asks the browser to store a cookie and send it back with later requests.
Cache-Control	How long the response may be reused without asking again.

HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 131
Cache-Control: max-age=600

<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Hello</title></head>
<body><h1>Hello, web!</h1></body>
</html>
```

Example. The sample response says: it worked (200 OK); the body is HTML in UTF-8, 131 bytes long, reusable for 600 seconds. An API answering GET /api/products/42 sends the same structure with Content-Type: application/json and a body such as {"id":42,"name":"Running shoes","price":89.9}.

Common mistake. Judging the result by the body alone. A server can send a friendly page with status 404, and a badly written API can send 200 OK with {"error":"not found"}. Code that reads a response checks the **status code first**, and a well-designed API treats "200 with an error body" as a bug.

Status codes

Status code families

Every response carries a three-digit **status code** whose first digit gives the family: **1xx** informational, **2xx** success, **3xx** redirection, **4xx** client error, **5xx** server error.

The five families

Family	Meaning	Who acts next	Typical codes
1xx	Informational: received, still working	Nobody; you rarely see these	100 Continue
2xx	Success	Nobody: use the body	200, 201, 204
3xx	Redirection, or "use your cached copy"	The browser, automatically	301, 302, 304
4xx	Client error: a bad or forbidden request	The client: fix the request	400, 401, 403, 404
5xx	Server error: the server failed	The server's developers	500, 503

Example. Your page loads but has no styles. In the Network tab, styles.css shows **404**: the server is fine and the path in your <link href="..."> is wrong, so the fix is in your HTML. A **500** instead would mean the HTML is fine and the bug is in the server code.

Common mistake. "4xx means the user typed something wrong." It means the **request** was wrong, and requests are often built by your own code: a misspelled href, a missing form field, a login token not sent. "Client error" names the side of the conversation, not the person.

The status codes you will use

About a dozen status codes cover almost every situation, best learned in pairs that are easy to confuse: **200/201, 301/302, 401/403, 404/500**.

Common codes

Code	Meaning	Typical situation
200 OK	Success, here is the body	A page, or a GET on an API
201 Created	A new resource was created	A successful POST; Location points to it
204 No Content	Success, nothing to send back	A successful DELETE
301 Moved Permanently	The resource has a new URL for good	A site moved from http:// to https://
302 Found	Temporarily at another URL	After logging in, go to the dashboard
304 Not Modified	Your cached copy is still valid	Reloading a page whose CSS did not change
400 Bad Request	The request is malformed or invalid	A required field is missing
401 Unauthorized	Not authenticated	No login token sent, or it expired
403 Forbidden	Authenticated, but not allowed	A regular user opens the admin panel
404 Not Found	Nothing at that URL	A misspelled file name in an href
500 Internal Server Error	The server code failed	An error the server code did not handle
503 Service Unavailable	The server cannot answer right now	Maintenance or overload

- **401 is badly named:** it means **unauthenticated** ("log in first"), while **403** means "I know who you are, and you may not" (see [Authentication vs authorisation](#)).
- **304 is not a redirect:** it is in the 3xx family, but it tells the browser its cached copy is still valid, so no body is sent.

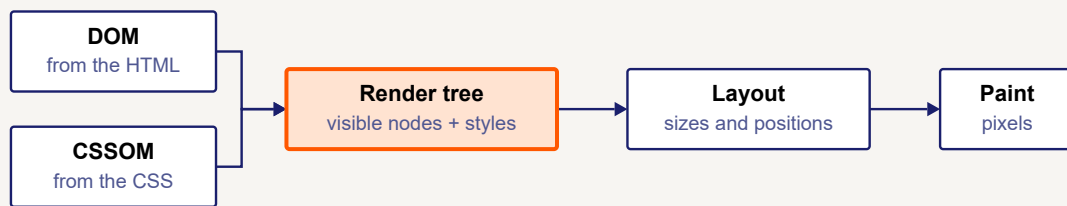
Example. A task API: GET /tasks without logging in returns **401**; logged in as a regular user, DELETE /tasks/3 (someone else's task) returns **403**; GET /tasks/999 returns **404**; a valid POST /tasks returns **201** with Location: /tasks/8; a typo in the server code that throws an error returns **500**.

Common mistake. Answering or expecting 500 when a resource does not exist. Nothing failed on the server: the client asked for something that is not there, so the right answer is 404. A 500 always means a bug or crash in the server.

In the browser: rendering and DevTools

From bytes to pixels: how the browser renders

The browser parses HTML into the **DOM** and CSS into the **CSSOM**, combines them into a **render tree**, computes the **layout** of every box and **paints** the pixels, repeating the affected steps whenever the page changes.



Two models merge into one render tree; then layout and paint.

Render tree Only the nodes that will be shown, each with its computed styles: an element with display: none is in the DOM but not here.

Layout The exact size and position of every box, for the current window width.

Paint The pixels. When JavaScript changes the DOM, or the window is resized, the browser redoes the affected steps.

Example. A page links site.css in its <head> and shows a big image. The browser builds the DOM, requests site.css and the image in parallel, waits for the CSS (it does not paint unstyled content), lays out the boxes and paints. When the image arrives its box is filled; if the HTML gave the image no width and height, the layout is redone and the text below **jumps**.

Common mistake. Thinking JavaScript edits the HTML file. The file on the server never changes; JavaScript edits the **DOM**, the live copy in memory. That is why View source (the file as it arrived) and the DevTools Elements tab (the current DOM) can differ, and why a reload brings the original back.

Watching the conversation: the Network tab

The **Network** tab of the browser DevTools lists every request a page makes, with its method, status code, type, size and time, and shows the headers and body of each one.

1. Open DevTools: F12, or Ctrl+Shift+I (Cmd+Opt+I on macOS).
2. Choose **Network**.
3. **Reload** the page: the tab only records while it is open.
4. Click a row: **Headers** shows the request line, the status and both sets of headers; **Response** or **Preview** shows the body.

Disable cache See fresh 200 responses instead of cached ones.

Type filters Doc, CSS, JS, Img, Fetch/XHR: show one kind of request.

Red rows Failed requests: 4xx, 5xx or network errors.

Example. Open a news site with the Network tab open and reload. The first row is the document itself (type document, status 200, Content-Type: text/html); then come dozens of rows for CSS, JavaScript, images and fonts, many from other hosts. If your own page shows a red row `logo.png 404`, the Headers panel shows the exact URL requested, so you can see which part of your `src` path is wrong.

Common mistake. Opening the Network tab after the page has loaded and finding it empty. The tab only records while DevTools is open: open it first, then reload.

The big picture

Front end and back end

The **front end** is the code that runs in the user's browser (HTML, CSS, JavaScript); the **back end** is the code that runs on the server, together with the database.



The browser asks; only the server touches the data.

The two halves

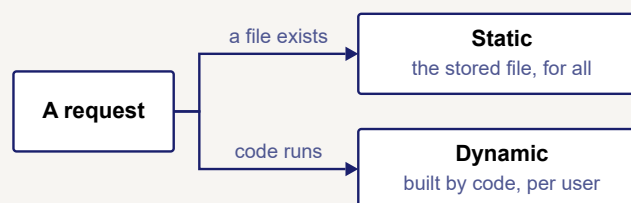
	Front end	Back end
Runs on	The user's browser	The server
Typical tools	HTML, CSS, JavaScript, often a front-end framework	A server runtime such as Node.js, and a database
Good at	Drawing the page, reacting instantly to the user	Storing data, keeping secrets, enforcing rules
Cannot	Be trusted: the user controls it	Change the page directly: it only answers requests

Example. Adding a task in a to-do app: the front end checks that the title is not empty and shows an error at once (a good experience), then sends `POST /api/tasks`. The back end checks the title **again**, saves it in the database and answers `201 Created` with the new task as JSON, and the front end adds it to the list on screen.

Common mistake. Trusting checks done only in the front end. Anyone can open DevTools and delete a required attribute, or skip your page and send the request with `curl`. Every rule that matters (prices, permissions, validation) is checked again in the back end.

Static and dynamic

A **static** response is a file sent exactly as it is stored, the same for everyone; a **dynamic** response is generated by server code for each request, usually from a database.



The question is who builds the response: a stored file, or code at that moment.

Example. A small football club site (home, fixtures, squad, tickets) is static: you can open `fixtures.html` straight from your disk (`file:///...`) without any server. A "My tickets" page listing what the logged-in member bought cannot be static: the server must look the member up in a database, which is a job for a back end (see [Server-side JavaScript](#)).

Common mistake. Calling a page "dynamic" because it has animations or a drop-down menu. Those run in the browser; if the server sends the same file to everyone, the site is static.

From static pages to a full-stack app

A web application is built in layers: a **static site** (HTML and CSS), made **interactive** with browser JavaScript, backed by a **server API** that answers with JSON and stores data in a **database**, and often topped by a **front-end application** that redraws the page from that data.

The layers of a typical web application

Layer	What it adds	Typical tools	Runs on
Static site	Content and structure, then presentation and layout	Semantic HTML, CSS (box model, Flexbox, Grid)	Browser
Interactive front end	Behaviour: reacting to clicks, validating forms, changing the page	JavaScript and the DOM	Browser
Server and API	Shared data, rules and secrets, answered as JSON over HTTP	A server runtime and a web framework, REST design, authentication	Server
Database	Data that survives restarts and is shared by every user	A relational database (SQL) or a document database	Database server
Front-end application	One page that fetches data from the API and redraws only what changed	Components, state and client-side routing	Browser

Front-end developer The browser side: HTML, CSS, JavaScript, a front-end framework, accessibility and performance.

Back-end developer The server side: API design, business rules, databases, authentication, security.

Full-stack developer Both halves, and above all the HTTP contract between them: which URLs exist, what JSON they accept and return, and which status codes they answer.

Example. A task tracker grows layer by layer: a static `tasks.html` with a form; JavaScript that checks the title is not empty and adds the task to the list; an API where `POST /api/tasks` answers `201 Created` and `GET /api/tasks/999` answers `404`; a database, so the tasks survive a restart and every device sees the same list; finally a front-end app that adds the new task without reloading the page.

Common mistake. Treating "how the web works" as theory you can skip. Every later layer assumes it: server frameworks, API design and authentication all talk about headers, status lines, JSON bodies and statelessness without introducing them again.